Vehicle Rental Management System

Final Project Documentation - Database Systems

Field	Details
Project Title	Vehicle Rental Management System (VRMS)
Course	Database Systems
Program / Section	BSCS - Section B
Instructor	Ms. Ambreen Akmal
Institution	The University of Lahore
Submission Scope	Phase I and Phase II
Database / Tool	PostgreSQL 15+ / pgAdmin 4

Group Members

Name
Muhammad Usman Tariq
Fazal Shahbaz
Fahad Zubair

Main GitHub Repository: https://github.com/70191-dev/Vehicle-Rental-Management-System/

1. Project Scenario Description

The Vehicle Rental Management System (VRMS) is a relational database project that automates the operations of a vehicle rental agency. It maintains customer records, vehicle inventory, bookings, payments, and return details in a structured PostgreSQL database.

The system replaces manual ledgers and disconnected spreadsheets with a central database that reduces duplicate bookings, protects payment history, tracks vehicle availability, and supports useful business reports such as customer rental history and revenue by vehicle.

2. Problem Statement

- Manual rental records can cause double booking of vehicles for overlapping dates.
- Payment records may be lost or disputed when they are not linked to bookings.
- Vehicle availability is difficult to monitor without a current inventory status.
- Revenue reporting is slow when totals must be calculated by hand.
- Return condition and late fee records need an auditable database trail.

3. Database Design Overview

The database is normalized around five main tables. Each table has a primary key and the transactional tables are connected through foreign keys so the rental lifecycle remains consistent from booking to payment and return.

Table	Primary Key	Purpose
Customer	CustomerID	Stores customer personal and contact details.
Vehicle	VehicleID	Stores the vehicle fleet, rates, plates, and status.
Booking	BookingID	Links a customer with a vehicle for a rental period.
Payment	PaymentID	Records payments against bookings.
Return	ReturnID	Records actual return date, condition, and late fee.

4. Relationships

Relationship	Tables	Cardinality	Business Meaning
Makes	Customer to Booking	1 : N	One customer can create many bookings.
For	Vehicle to Booking	1 : N	One vehicle can appear in many bookings over time.
Has Payment	Booking to Payment	1 : N	A booking can have multiple installment

			payments.
Ends With	Booking to Return	1 : 1	A completed booking has one return record.

Foreign keys used: Booking.CustomerID references Customer.CustomerID, Booking.VehicleID references Vehicle.VehicleID, Payment.BookingID references Booking.BookingID, and Return.BookingID references Booking.BookingID.

5. ERD and EERD

The ERD represents the five core entities and their relationships. The EERD extends the model with specialization: Customer is divided into Individual Customer and Corporate Customer, while Vehicle is divided into Car, Bike, and Truck.

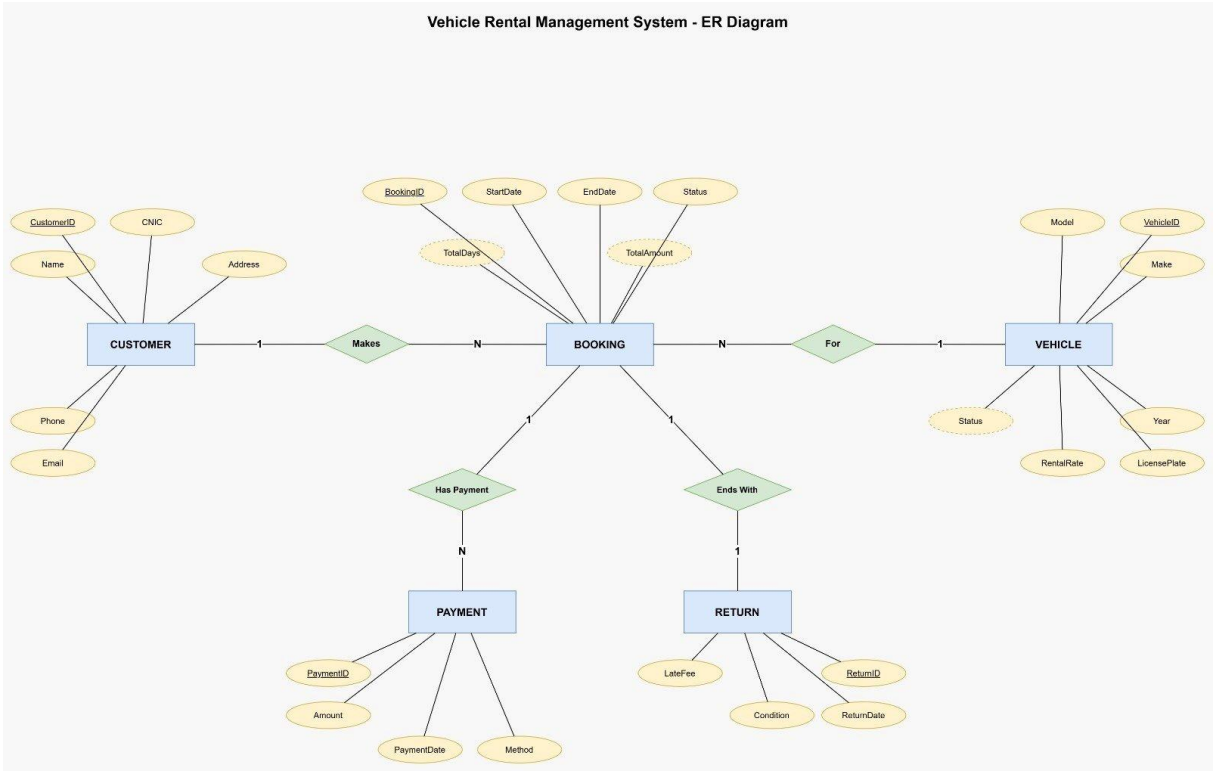


Figure 1: Entity Relationship Diagram (ERD)

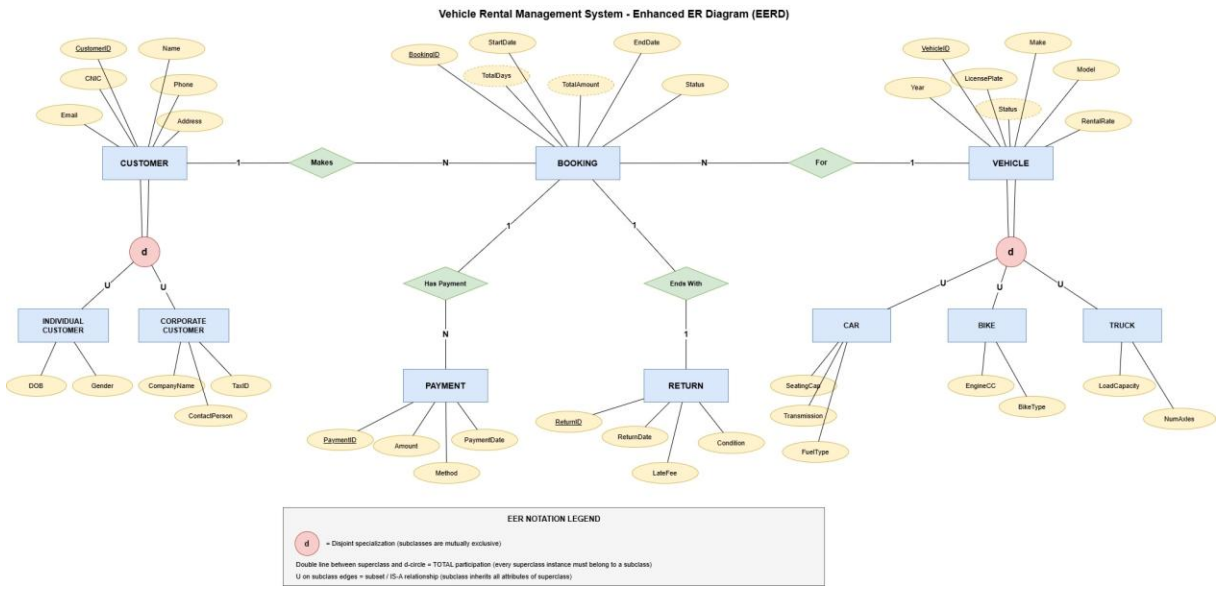


Figure 2: Enhanced Entity Relationship Diagram (EERD)

6. Phase 2 Functional Features

Phase 2 adds ten runnable SQL features in sql/04_features.sql. These features demonstrate real operations in the rental business and cover DML, DDL, and DCL requirements.

#	Feature	SQL Type	Output / Purpose
1	Register a New Customer	INSERT	Adds a new customer record.
2	Add a New Vehicle	INSERT	Adds a new rentable vehicle to inventory.
3	Create a New Booking	INSERT	Creates a rental booking for a customer and vehicle.
4	Search Available Vehicles	SELECT + WHERE	Filters available vehicles within a rate range.
5	View Customer Rental History	SELECT + JOIN	Shows a customer's bookings with vehicle details.
6	Record a Payment	INSERT	Stores payment details for a booking.
7	Update Vehicle Status	UPDATE	Marks a vehicle as rented, available, or maintenance.
8	Process Vehicle Return	INSERT + UPDATE	Logs return condition, late fee, and completes booking.
9	Cancel / Delete a Booking	DELETE	Removes cancelled bookings.
10	Revenue Report by Vehicle	SELECT + JOIN + GROUP	Aggregates total revenue per vehicle.

		BY	

7. SQL Command Categories Covered

7.1 DDL - Data Definition Language

- CREATE TABLE statements define Customer, Vehicle, Booking, Payment, and Return in sql/01_schema.sql.
- ALTER TABLE adds RegistrationDate to Customer.
- CREATE OR REPLACE VIEW creates ActiveRentals for current active bookings.
- CREATE INDEX creates idx_vehicle_status to speed up vehicle status searches.

7.2 DML - Data Manipulation Language

- INSERT is used for customers, vehicles, bookings, payments, and returns.
- SELECT is used for searches, rental history, active rental views, and revenue reports.
- UPDATE is used to change booking and vehicle statuses.
- DELETE is used to remove cancelled bookings.

7.3 DCL - Data Control Language

- CREATE ROLE creates the rental_clerk database role.
- GRANT gives SELECT permission on project tables and temporarily allows INSERT on Booking.
- REVOKE removes the INSERT permission from Booking to demonstrate permission control.

8. SQL Script Execution Order

1. Run sql/01_schema.sql to create the tables and constraints.
2. Run sql/02_sample_data.sql to insert the sample records.
3. Run sql/03_joins.sql to test Phase 1 join queries.
4. Run sql/04_features.sql to execute the ten Phase 2 features and DDL/DCL examples.

9. Key SQL Evidence

The complete SQL is stored in the repository. The representative examples below show the important commands used in Phase 2.

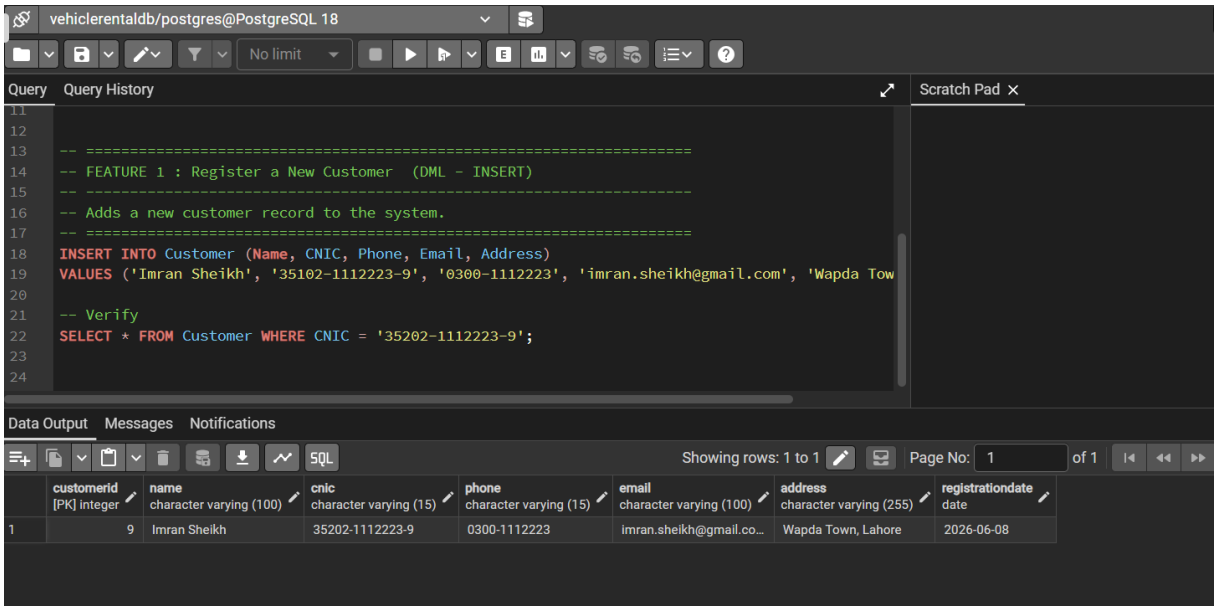
```
INSERT INTO Customer (Name, CNIC, Phone, Email, Address)
VALUES ('Imran Sheikh', '35202-1112223-9', '0300-1112223',
        'imran.sheikh@gmail.com', 'Wapda Town, Lahore');

SELECT c.Name AS Customer, b.BookingID, (v.Make || ' ' || v.Model) AS Vehicle,
       b.StartDate, b.EndDate, b.TotalAmount, b.Status
FROM Booking b
JOIN Customer c ON b.CustomerID = c.CustomerID
JOIN Vehicle v ON b.VehicleID = v.VehicleID
WHERE c.CustomerID = 1;
```

```
CREATE ROLE rental_clerk LOGIN PASSWORD 'clerk123';
GRANT SELECT ON Customer, Vehicle, Booking, Payment, "Return" TO rental_clerk;
REVOKE INSERT ON Booking FROM rental_clerk;
```

10. Screenshots of Feature Outputs

The following screenshots were captured from pgAdmin after running each feature in sql/04_features.sql.

Feature 1: Register a New Customer



Screenshot 1: Feature 1: Register a New Customer

Feature 2: Add a New Vehicle

vehiclerentaldb/postgres@PostgreSQL 18

No limit

Query | Query History | Scratch

```
-- ========================================================================
-- FEATURE 2 : Add a New Vehicle to Inventory  (DML - INSERT)
-- ========================================================================
-- Adds a new rentable vehicle to the fleet.
-- ========================================================================
INSERT INTO Vehicle (Make, Model, Year, LicensePlate, RentalRate, Status)
VALUES ('Toyota', 'Yaris', 2025, 'LEX-2025', 4800.00, 'Available');

-- Verify
SELECT * FROM Vehicle WHERE LicensePlate = 'LEX-2024';
```

Data Output | Messages | Notifications

Showing rows: 1 to 1 | Page No:

	vehicleid [PK] integer	make character varying (50)	model character varying (50)	year integer	licenseplate character varying (20)	rentalrate numeric (10,2)	status character varying (20)
1	9	Toyota	Yaris	2024	LEX-2024	4800.00	Available

Screenshot 2: Feature 2: Add a New Vehicle

Feature 3: Create a New Booking

vehiclerentaldb/postgres@PostgreSQL 18

No limit

Query | Query History

```
1
2  -- ======================================================================
3  -- FEATURE 3 : Create a New Booking  (DML - INSERT)
4  -- ----------------------------------------------------------------------
5  -- Creates a rental booking linking a customer to a vehicle.
6  -- ======================================================================
7  INSERT INTO Booking (CustomerID, VehicleID, StartDate, EndDate, Status, TotalAmount)
8  VALUES (3, 5, '2026-06-01', '2026-06-04', 'Active', 18000.00);
9
10 -- Verify
11 SELECT * FROM Booking WHERE CustomerID = 3 AND VehicleID = 5;
12
13
```

Data Output | Messages | Notifications

Showing rows: 1 to 2

	bookingid [PK] integer	customerid integer	vehicleid integer	startdate date	enddate date	status character varying (20)	totalamount numeric (10,2)
1	9	3	5	2026-06-01	2026-06-...	Active	18000.00
2	11	3	5	2026-06-01	2026-06-...	Active	18000.00

Screenshot 3: Feature 3: Create a New Booking

Feature 4: Search / Filter Available Vehicles

Query | Query History | Scratch Pad

```
-- ====================================================================
-- FEATURE 4 : Search / Filter Available Vehicles  (DML - SELECT + WHERE)
-- --------------------------------------------------------------------
-- Finds all vehicles currently available for rent, optionally within
-- a price range.
-- ====================================================================
SELECT VehicleID, Make, Model, Year, RentalRate
FROM Vehicle
WHERE Status = 'Available'
  AND RentalRate BETWEEN 3000 AND 8000
ORDER BY RentalRate ASC;
```

Data Output | Messages | Notifications

Showing rows: 1 to 4 | Page No: 1

	vehicleid [PK] integer	make character varying (50)	model character varying (50)	year integer	rentalrate numeric (10,2)
1	3	Suzuki	Cultus	2021	3500.00
2	9	Toyota	Yaris	2024	4800.00
3	12	Toyota	Yaris	2025	4800.00
4	1	Toyota	Corolla GLi	2022	5500.00

Screenshot 4: Feature 4: Search / Filter Available Vehicles

Feature 5: View Customer Rental History

Query | Query History | Scratch

```
-- ========================================================================
-- FEATURE 5 : View Customer Rental History  (DML - SELECT + JOIN)
-- ------------------------------------------------------------------------
-- Shows the full booking history of a specific customer.
-- ========================================================================
SELECT
    c.Name          AS Customer,
    b.BookingID,
    (v.Make || ' ' || v.Model) AS Vehicle,
    b.StartDate,
    b.EndDate,
    b.TotalAmount,
    b.Status
```

Data Output | Messages | Notifications

Showing rows: 1 to 2 | Page No:

	customer character varying (100)	bookingid integer	vehicle text	startdate date	enddate date	totalamount numeric (10,2)	status character varying (20)
1	Ahmed Raza	5	Honda City	2026-05-05	2026-05-...	12000.00	Completed
2	Ahmed Raza	1	Honda Civ...	2026-04-01	2026-04-...	30000.00	Completed

Screenshot 5: Feature 5: View Customer Rental History

Feature 6: Record a Payment

vehiclerentaldb/postgres@PostgreSQL 18

No limit

Query | Query History

```
1
2
3   -- ============================================================
4   -- FEATURE 6 : Record a Payment  (DML - INSERT)
5   -- ------------------------------------------------------------
6   -- Records a payment made against a booking.
7   -- ============================================================
8   INSERT INTO Payment (BookingID, Amount, PaymentDate, Method)
9   VALUES (6, 7000.00, '2026-05-09', 'Cash');
10
11  -- Verify
12  SELECT * FROM Payment WHERE BookingID = 6;
13
14
```

Data Output | Messages | Notifications

SQL

Showing rows: 1

	paymentid [PK] integer	bookingid integer	amount numeric (10,2)	paymentdate date	method character varying (20)
1	10	6	7000.00	2026-05-09	Cash
2	11	6	7000.00	2026-05-09	Cash

Screenshot 6: Feature 6: Record a Payment

Feature 7: Update Vehicle Status

Query | Query History

```
-- ================================================================
-- FEATURE 7 : Update Vehicle Status  (DML - UPDATE)
-- ----------------------------------------------------------------
-- Marks a vehicle as 'Rented' when it is booked, or back to
-- 'Available' / 'Maintenance' as needed.
-- ================================================================
UPDATE Vehicle
SET Status = 'Rented'
WHERE VehicleID = 5;

-- Verify
SELECT VehicleID, Make, Model, Status FROM Vehicle WHERE VehicleID = 5;
```

Data Output | Messages | Notifications

Showing rows: 1 to 1

	vehicleid [PK] integer	make character varying (50)	model character varying (50)	status character varying (20)
1	5	Honda	City	Rented

Screenshot 7: Feature 7: Update Vehicle Status

Feature 8: Process a Vehicle Return + Late Fee

Query | Query History

```
-- FEATURE 8 : Process a Vehicle Return + Late Fee  (DML - INSERT + UPDATE)
-- ------------------------------------------------------------------------
-- Logs a vehicle return, applies a late fee, and frees up the vehicle.
-- ========================================================================
INSERT INTO "Return" (BookingID, ReturnDate, "Condition", LateFee)
VALUES (4, '2026-04-16', 'Good', 15000.00);

-- Mark the booking completed and free the vehicle
UPDATE Booking SET Status = 'Completed' WHERE BookingID = 2;
UPDATE Vehicle SET Status = 'Available'
WHERE VehicleID = (SELECT VehicleID FROM Booking WHERE BookingID = 2);

-- Verify
SELECT * FROM "Return" WHERE BookingID = 2;
```

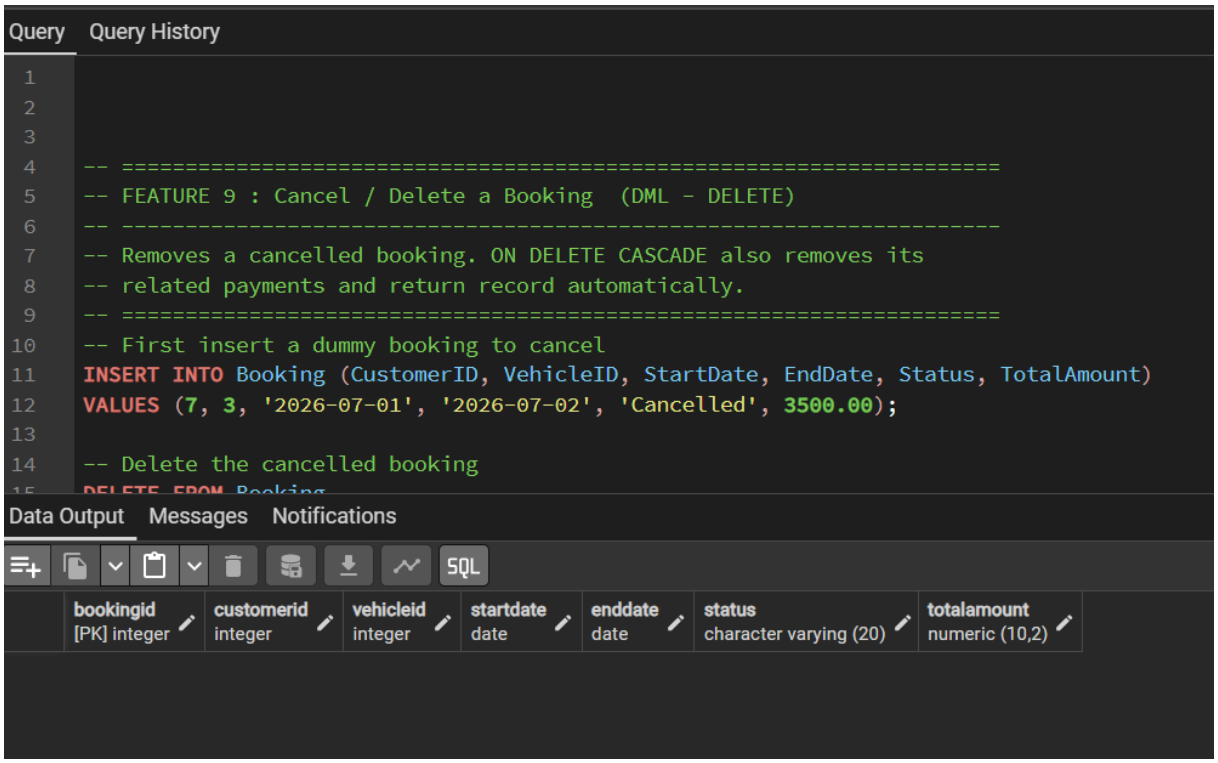
Data Output | Messages | Notifications

SQL | Showing rows: 1 to 1

	returnid [PK] integer	bookingid integer	returndate date	Condition character varying (50)	latefee numeric (10,2)
1	5	2	2026-04-16	Good	15000.00

Screenshot 8: Feature 8: Process a Vehicle Return + Late Fee

Feature 9: Cancel / Delete a Booking



Screenshot 9: Feature 9: Cancel / Delete a Booking

Feature 10: Revenue Report by Vehicle

vehiclerentaldb/postgres@PostgreSQL 18

No limit

Query | Query History

```
1
2
3   -- ============================================================================
4   -- FEATURE 10 : Revenue Report by Vehicle  (DML - SELECT + JOIN + GROUP BY)
5   -- ----------------------------------------------------------------------------
6   -- Aggregates total revenue earned per vehicle (business reporting).
7   -- ============================================================================
8   SELECT
9       v.VehicleID,
10      (v.Make || ' ' || v.Model) AS Vehicle,
11      COUNT(b.BookingID)         AS TimesBooked,
12      COALESCE(SUM(b.TotalAmount), 0) AS TotalRevenue
13  FROM Vehicle v
14  LEFT JOIN Booking b ON v.VehicleID = b.VehicleID
15  GROUP BY v.VehicleID, v.Make, v.Model
```

Data Output | Messages | Notifications

SQL

Showing rows: 1 to 1

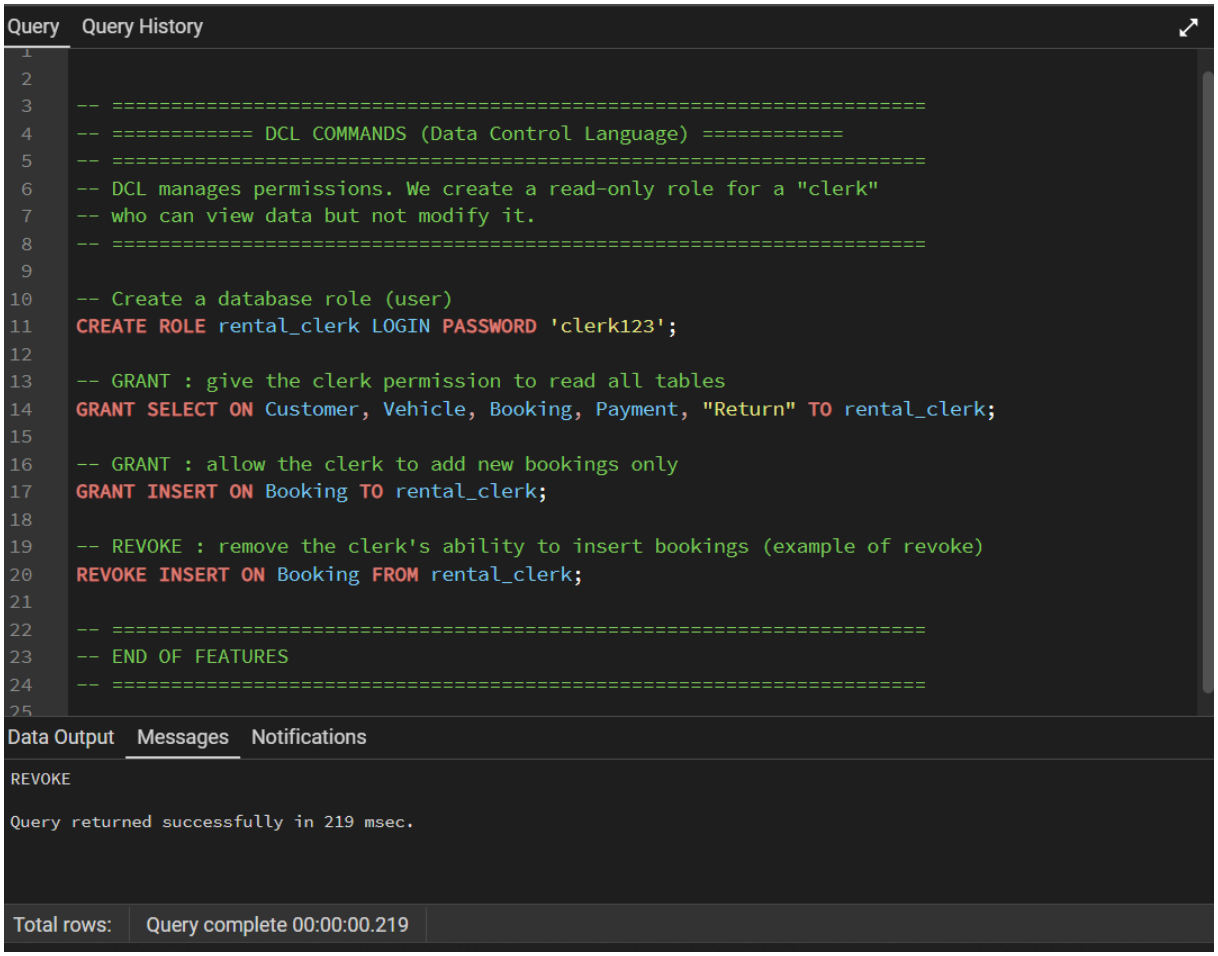
	vehicleid [PK] integer	vehicle text	timesbooked bigint	totalrevenue numeric
1	8	KIA Sportage	1	77000.00
2	4	Toyota Fortuner	1	75000.00
3	2	Honda Civic	2	67500.00
4	5	Honda City	3	48000.00
5	6	Haval H6 Facelift	1	24000.00
6	1	Toyota Corolla ...	1	11000.00
7	3	Suzuki Cultus	1	7000.00
8	7	Suzuki Alto	0	0
9	12	Toyota Yaris	0	0

Total rows: 10 | Query complete 00:00:00.267

Screenshot 10: Feature 10: Revenue Report by Vehicle

11. DCL Output Evidence

This screenshot verifies the DCL section that creates, grants, and revokes permissions for the rental_clerk role.



Screenshot 11: DCL role and permission commands

12. Conclusion

The Vehicle Rental Management System demonstrates a complete relational database workflow for a realistic rental business. The project includes normalized schema design, ER/EER modeling, sample data, join queries, ten Phase 2 functional SQL features, DDL/DML/DCL command coverage, and pgAdmin screenshots proving the query outputs.